

**SOFTWARE REQUIREMENTS SPECIFICATION  
VERSION 1.0**

**YADA  
(YET ANOTHER DELIVERY APP)**

**JUNE 28<sup>TH</sup>, 2026**

## REVISION HISTORY

| Date         | Version | Description | Author              |
|--------------|---------|-------------|---------------------|
| <28/06/2026> | <1.0>   | SRS 1.0     | Flavio N. B. Sobbin |

## TABLE OF CONTENTS

|         |                                                 |    |
|---------|-------------------------------------------------|----|
| 1.      | Introduction.....                               | 1  |
| 1.1.    | Purpose.....                                    | 1  |
| 1.2.    | Glossary.....                                   | 1  |
| 1.3.    | Intended Audience and Reading Suggestions ..... | 1  |
| 1.4.    | Project Scope.....                              | 1  |
| 2.      | Overall Description.....                        | 2  |
| 2.1.    | System Environment .....                        | 2  |
| 2.2.    | Functional Requirements Specification .....     | 2  |
| 2.2.1   | Business Use Cases.....                         | 2  |
| 2.2.1.1 | Business Use Case: Register Account .....       | 2  |
| 2.2.1.2 | Business Use Case: Request Courier .....        | 3  |
| 2.2.1.3 | Business Use Case: Track Delivery .....         | 3  |
| 2.2.1.4 | Business Use Case: Cancel Request.....          | 4  |
| 2.2.1.5 | Business Use Case: Rate Courier .....           | 4  |
| 2.2.2   | Courier Use Cases.....                          | 5  |
| 2.2.2.1 | Courier Use Case: Register Account .....        | 5  |
| 2.2.2.2 | Courier Use Case: Accept Request.....           | 6  |
| 2.2.2.3 | Courier Use Case: Set Availability .....        | 6  |
| 2.2.2.4 | Courier Use Case: Update Trip Status.....       | 6  |
| 2.2.2.5 | Courier Use Case: Rate Business .....           | 7  |
| 2.3.    | User Characteristics.....                       | 7  |
| 2.3.1   | Business .....                                  | 7  |
| 2.3.2   | Courier .....                                   | 8  |
| 2.4.    | Operating Environment.....                      | 8  |
| 2.5.    | Design and Implementation Constraints .....     | 8  |
| 2.6.    | Assumptions and Dependencies.....               | 8  |
| 3.      | FUNCTIONAL REQUIREMENTS .....                   | 9  |
| 3.1.    | User Registration & Authentication .....        | 9  |
| 3.2.    | Ride Request & Matching.....                    | 9  |
| 3.3.    | Trip Lifecycle .....                            | 9  |
| 3.4.    | Ratings & Feedback .....                        | 9  |
| 3.5     | Provide Customer Support (Future) .....         | 10 |
| 4.      | External Interface Requirements.....            | 11 |
| 4.1.    | User Interfaces.....                            | 11 |

|      |                                   |    |
|------|-----------------------------------|----|
| 4.2. | Hardware Interfaces .....         | 11 |
| 4.3. | Software Interfaces .....         | 11 |
| 4.4. | Communications Interfaces .....   | 11 |
| 5.   | Non-functional Requirements ..... | 12 |
| 5.1. | Performance Requirements .....    | 12 |
| 5.2. | Security Requirements .....       | 12 |
| 5.3. | Scalability Requirements .....    | 12 |
| 5.4. | Usability .....                   | 12 |
| 5.5. | Privacy & Compliance .....        | 12 |
| 5.6. | Compatibility .....               | 13 |

## 1. INTRODUCTION

### 1.1. Purpose

The purpose of this document is to present a detailed description of the YADA web application. It will explain the purpose and features of the system, the interfaces of the system, what the system will do, the constraints under which it must operate and how the system will react to external stimuli. This document is intended for both clients and developers of the system.

### 1.2. Glossary

This document contains the following conventions.

| Term     | Definition                                                                                       |
|----------|--------------------------------------------------------------------------------------------------|
| User     | A registered person in the system. (a person with a user account)                                |
| Business | The user that requests for a courier.                                                            |
| Courier  | The user to deliver the product.                                                                 |
| HTTPS    | HyperText Transfer Protocol Secure                                                               |
| YADA     | Yet Another Delivery App - the motor courier request web application described in this document. |
| Trip     | A single delivery job, from request through completion or cancellation.                          |
| OTP      | One-Time Password - a single-use code sent by SMS or email to verify a user.                     |
| OAuth    | An open standard that allows users to sign in using a third-party account.                       |
| ETA      | Estimated Time of Arrival.                                                                       |
| API      | Application Programming Interface.                                                               |
| SMS      | Short Message Service.                                                                           |
| SMTP     | Simple Mail Transfer Protocol - used for sending account recovery emails.                        |
| TLS      | Transport Layer Security - cryptographic protocol that secures data in transit.                  |
| HTML5    | Fifth version of the HyperText Markup Language, required of supported web browsers.              |

### 1.3. Intended Audience and Reading Suggestions

This document targets the following class of individuals:

- Food businesses that rely heavily on motor delivery services.
- Motor couriers in search for food delivery servicing.

### 1.4. Project Scope

This software will be a Motor Courier Request Web Application targeting food businesses. This system will be designed to streamline delivery orders, which would otherwise have to be done manually through text messaging and calls. This makes it easy to request for and track delivery services in one space.

## 2. OVERALL DESCRIPTION

### 2.1. System Environment

YADA has two active actors; the Business and the Courier. Both actors access the system directly and at the same time.

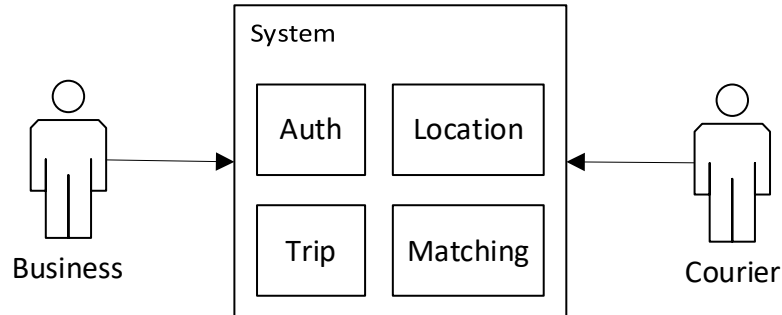


Figure 1: System Environment

### 2.2. Functional Requirements Specification

This section outlines the use cases for the user.

#### 2.2.1 Business Use Cases

The Business has the following set of use cases:

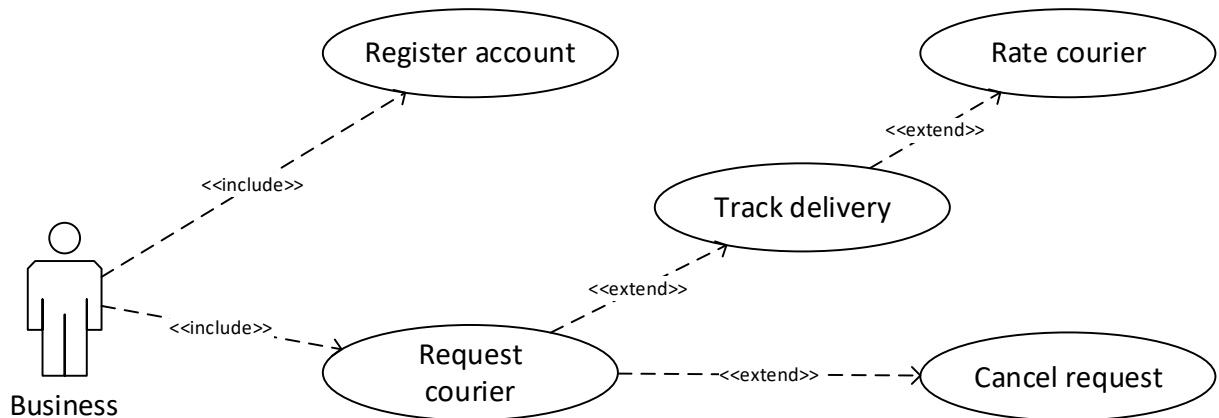
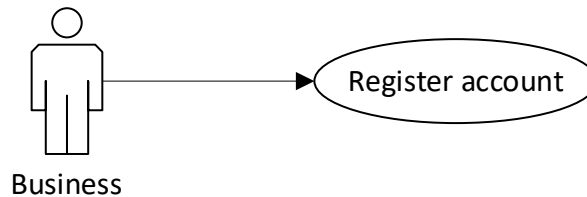


Figure 2: Business Use Cases

##### 2.2.1.1 Business Use Case: Register Account

**Diagram:**



**Brief Description**

The User creates an account by providing their details and verifying their contact information.

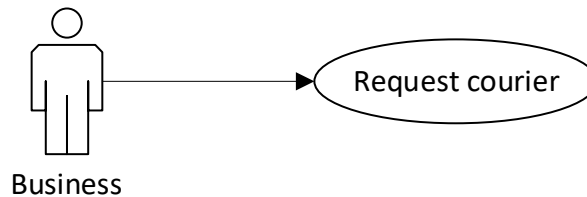
### **Initial Step-By-Step Description**

Before this use case can be initiated, the User has already opened the YADA registration page.

1. The User enters their phone number, email, and password, or selects third-party OAuth.
2. The User submits the registration form.
3. The User enters the one-time password (OTP) sent to their phone or email.
4. The System confirms the account and redirects the User to the dashboard.

### **2.2.1.2 Business Use Case: Request Courier**

#### **Diagram:**



#### **Brief Description**

The User enters the delivery destination and clicks search.

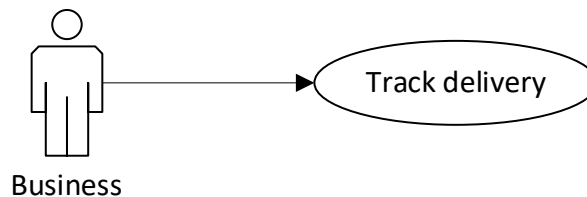
### **Initial Step-By-Step Description**

Before this use case can be initiated, the User has already accessed the dashboard.

1. The User enters the delivery destination.
2. The User clicks the search.

### **2.2.1.3 Business Use Case: Track Delivery**

#### **Diagram:**



#### **Brief Description**

The User views the assigned courier's live location and estimated time of arrival for an active trip.

### **Initial Step-By-Step Description**

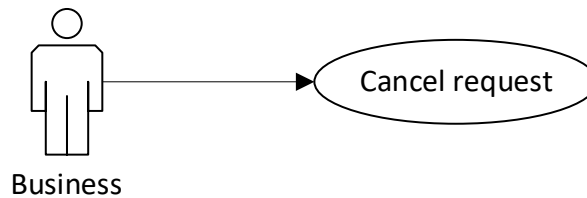
Before this use case can be initiated, the User has an active trip that a courier has accepted.

1. The User opens the active trip from the dashboard.
2. The User views the courier's live position and ETA on the map.

3. The System updates the courier's location in near real time until the trip is completed.

#### 2.2.1.4 Business Use Case: Cancel Request

**Diagram:**



##### **Brief Description**

The User cancels a delivery request before the trip has started.

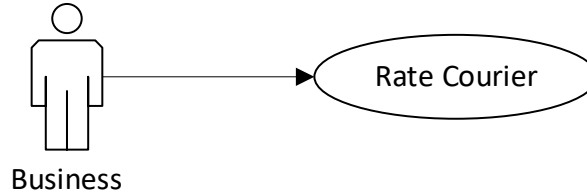
##### **Initial Step-By-Step Description**

Before this use case can be initiated, the User has a pending or accepted trip that has not yet started.

1. The User opens the pending trip from the dashboard.
2. The User selects the cancel option.
3. The System applies any configured cancellation rules and confirms the cancellation.

#### 2.2.1.5 Business Use Case: Rate Courier

**Diagram:**



##### **Brief Description**

The User rates the courier and leaves optional feedback after a completed trip.

##### **Initial Step-By-Step Description**

Before this use case can be initiated, the User has a completed trip that has not yet been rated.

1. The User selects a star rating from 1 to 5 for the courier.
2. The User optionally enters a written comment.
3. The User submits the rating.
4. The System records the rating and updates the courier's rolling average.



## 2.2.2 Courier Use Cases

The Courier has the following set of use cases:

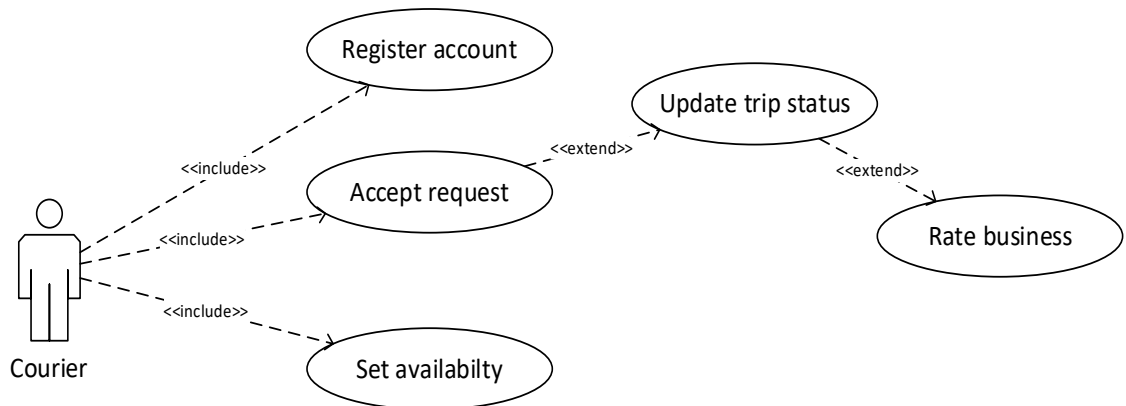
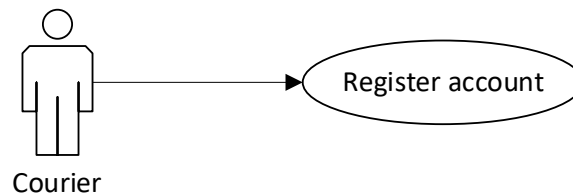


Figure 3: Courier Use Cases

### 2.2.2.1 Courier Use Case: Register Account

**Diagram:**



#### Brief Description

The Courier creates an account by submitting personal details and a profile photograph for verification.

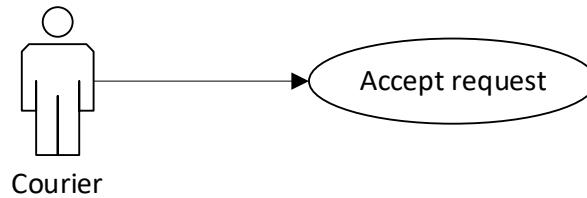
#### Initial Step-By-Step Description

Before this use case can be initiated, the Courier has opened the YADA registration page.

1. The Courier enters their phone number, email, and password, or selects third-party OAuth.
2. The Courier submits the registration form.
3. The Courier verifies their phone number using the one-time password (OTP) sent by SMS.
4. The Courier uploads a profile photograph.
5. The System creates the account and grants the Courier access to the dashboard.

### 2.2.2.2 Courier Use Case: Accept Request

#### Diagram:



#### Brief Description

The Courier reviews an incoming trip offer and accepts it to be assigned the delivery.

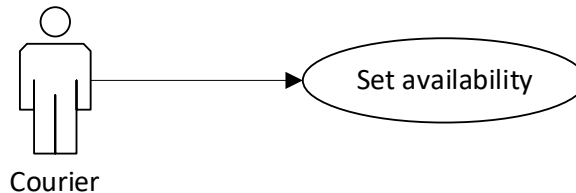
#### Initial Step-By-Step Description

Before this use case can be initiated, the Courier is online and available, and the System has dispatched a trip offer to the Courier.

1. The Courier receives a trip offer showing the pickup location, destination, and estimated fare.
2. The Courier reviews the offer details before the acceptance timeout expires.
3. The Courier accepts the offer.
4. The System assigns the trip to the Courier and notifies the Business.

### 2.2.2.3 Courier Use Case: Set Availability

#### Diagram:



#### Brief Description

The Courier toggles their availability so the System includes or excludes them from trip matching.

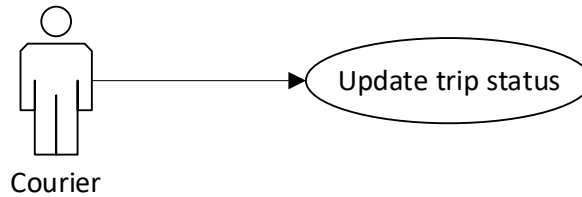
#### Initial Step-By-Step Description

Before this use case can be initiated, the Courier has signed in to the dashboard.

1. The Courier opens the availability toggle on the dashboard.
2. The Courier sets their status to online (available) or offline (unavailable).
3. The System updates the Courier's availability and includes them in matching only while online.

### 2.2.2.4 Courier Use Case: Update Trip Status

#### Diagram:



### **Brief Description**

The Courier advances the trip through its lifecycle states as the delivery progresses.

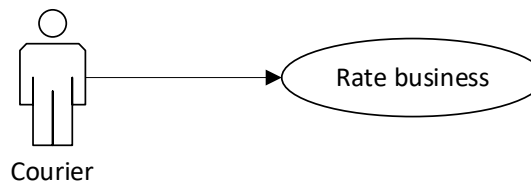
### **Initial Step-By-Step Description**

Before this use case can be initiated, the Courier has accepted a trip.

1. The Courier marks that they are heading to the pickup point.
2. The Courier marks arrival at the pickup point.
3. The Courier marks the trip as in progress after collecting the item.
4. The Courier marks the trip as completed upon delivery.
5. The System records each status change and updates the Business in near real time.

#### **2.2.2.5 Courier Use Case: Rate Business**

### **Diagram:**



### **Brief Description**

The Courier rates the Business and leaves optional feedback after a completed trip.

### **Initial Step-By-Step Description**

Before this use case can be initiated, the Courier has a completed trip that has not yet been rated.

1. The Courier selects a star rating from 1 to 5 for the Business.
2. The Courier optionally enters a written comment.
3. The Courier submits the rating.
4. The System records the rating and updates the Business's rolling average.

## **2.3. User Characteristics**

### **2.3.1 Business**

- Business is expected to be an internet literate.
- Business is expected to be a computer literate and to be able to navigate through sections with ease.

### 2.3.2 Courier

- Courier is expected to be an internet literate.
- Courier is expected to have basic knowledge of maps and route navigation.
- Courier is expected to be a smartphone literate and to be able to navigate through sections with ease.

## 2.4. Operating Environment

YADA is a browser-based web application that runs in a standard client-server environment. The client is served to any modern web browser with HTML5 and JavaScript support, on either desktop or mobile devices, and requires an active internet connection. The client communicates with a Node.js back-end over HTTPS, and all persistent data is held in a PostgreSQL database. Location-dependent features require the device's location services to be enabled, and the system relies on external providers for mapping and geolocation, SMS delivery (for OTP), and email (SMTP).

## 2.5. Design and Implementation Constraints

The design and implementation of YADA are subject to the following constraints:

- The system shall be delivered as a responsive web application accessed through a web browser; no native mobile application is provided in the initial release.
- All communication between the client and server shall take place over HTTPS using TLS 1.2 or higher.
- Real-time matching, tracking, and notifications depend on third-party mapping/geolocation and SMS gateway services, which must be reachable for those features to function.
- The application shall remain usable on low-bandwidth (3G) mobile networks.

## 2.6. Assumptions and Dependencies

The following assumptions and dependencies apply to YADA:

- It is assumed that businesses and couriers have access to an internet-capable device with a supported web browser.
- It is assumed that couriers use a smartphone with location services enabled and an active data connection while working.
- It is assumed that users grant the location permissions required for matching and live tracking.
- The system depends on the availability and reliability of external services, namely the SMS gateway (OTP), the email/SMTP provider, the mapping and geolocation provider, and any third-party OAuth providers.
- The system depends on its hosting infrastructure and network connectivity remaining available.

### 3. FUNCTIONAL REQUIREMENTS

#### 3.1. User Registration & Authentication

- The system shall allow users to register using a phone number, email, or third -party OAuth.
- The system shall verify users phone numbers via one-time password (OTP) sent by SMS.
- The system shall allow couriers to register by submitting personal details, and a profile photograph.
- The system shall authenticate all users via secure session tokens.
- The system shall allow users to reset credentials via OTP or email link.
- The system shall allow users to manage profile information (name, photo).

#### 3.2. Ride Request & Matching

- The system shall identify available couriers within a configurable radius of the pickup point using geospatial indexing.
- The system shall display an estimated time of arrival (ETA) before the business confirms a request.
- The system shall dispatch the trip offer to the highest-ranked courier (by proximity/ETA, headed destination, rating) with a configurable acceptance timeout.
- The system shall cascade the offer to the next-ranked courier if the offer is declined or times out.
- The system shall notify the business if no courier accepts within a configurable period and allow the business to retry or cancel.
- The system shall allow business to schedule trips in advance (optional/phase 2).

#### 3.3. Trip Lifecycle

- The system shall maintain each trip as a state machine: *requested* → *accepted* → *courier arriving* → *arrived* → *in progress* → *completed / cancelled*.
- The system shall show the business the assigned courier's name, photo, and rating upon acceptance.
- The system shall display the courier's live position and ETA to the business on a map, updated in near real time.
- The system shall allow either party to cancel a trip before it starts, applying cancellation rules where configured.
- The system shall record trip telemetry (route polyline, distance, duration, timestamps) for audit.

#### 3.4. Ratings & Feedback

- The system shall prompt both parties to rate each other (1–5 stars) with optional comments after each completed trip.
- The system shall compute and display rolling average ratings on user profiles.

- The system shall display demand heat maps or zone indicators to help drivers position themselves (optional/phase 2)

### **3.5 Provide Customer Support (Future)**

- The webapp shall provide feedback form for support and bug reports.
- The webapp shall provide a brief guide to navigate the UI.

## **4. EXTERNAL INTERFACE REQUIREMENTS**

### **4.1. User Interfaces**

- Front-end software: Svelte
- Back-end software: Node.js
- Database software: PostgreSQL

### **4.2. Hardware Interfaces**

The webapp is supported on any device that has access to the internet via a web browser in support. It is also supported by devices with location services. A smartphone is the most preferred device for couriers due to its portability.

### **4.3. Software Interfaces**

- A web browser with HTML5 and JavaScript support.
- A mobile phone with WebView capabilities.

### **4.4. Communications Interfaces**

- The system shall use the HTTPS protocol for communication over the internet.
- Emails shall be sent through SMTP for account password recovery.

## **5. NON-FUNCTIONAL REQUIREMENTS**

### **5.1. Performance Requirements**

- This system is web-based and has to be run from a web browser on a mobile or a desktop.
- The system shall complete business–courier matching (request to first courier offer) within 5 seconds.
- The system shall propagate courier location updates to the business’ map with end-to-end latency not exceeding 3 seconds.
- The system shall respond to 95% of API requests within 300 ms and 99% within 1 second under normal load.

### **5.2. Security Requirements**

- The system shall encrypt all data in transit using TLS 1.2 or higher.
- The system shall hash passwords with a modern adaptive algorithm (e.g., bcrypt/Argon2).
- The system shall enforce role-based access control (business, courier with least privilege).
- The system shall rate-limit authentication and OTP endpoints to mitigate brute-force and SMS-pumping attacks.

### **5.3. Scalability Requirements**

- The system shall partition geospatial and trip data such that adding new cities does not degrade performance in existing ones.
- The system shall handle at least 3 concurrent active trips and 3 location updates per second at launch, with a documented path to 10× that load.

### **5.4. Usability**

- A first-time business shall be able to complete registration and request a ride in under 3 minutes without assistance.
- The mobile view shall conform to platform accessibility guidelines (screen-reader support, minimum touch-target sizes, adequate contrast).
- The courier application shall remain legible and operable in bright sunlight and while vehicle-mounted (large controls, minimal interaction depth).
- The applications shall support localisation of language and currency per market (optional/phase 2).

### **5.5. Privacy & Compliance**

- The system shall collect location data only while required for service delivery and disclose this in the privacy policy.
- The system shall mask personal phone numbers in courier–driver communications.
- The system shall support account deletion requests within statutory timelines.



## 5.6. Compatibility

- Any device (smartphone preferred) with access to a supported web browser. This system will run on a supported web browser.
- The applications shall function on low-bandwidth mobile networks (3G) with graceful handling of intermittent connectivity.